QB

Quiz Battle

Real-Time Multiplayer Quiz System

Complete Project Report

Author	Manas Kumar
GitHub	MANAS-exe/QUIZ_BATTLE_SYSYTEM
Date	April 8, 2026
Version	1.0.0

Go 1.25 **Flutter 3.41** **Redis 7** **MongoDB 7**
RabbitMQ 3 **gRPC + Protobuf**

Contents

1 Project Overview **3**
1.1 Key Features . . . 3

2 Architecture & Technology Stack **4**
2.1 High-Level Architecture . . . 4
2.2 Technology Stack . . . 5
2.3 Technology Justifications . . . 5
2.3.1 gRPC over REST . . . 5
2.3.2 Redis for Live Game State . . . 5
2.3.3 RabbitMQ for Answer Processing . . . 6
2.3.4 Riverpod over BLoC/Provider . . . 6

3 System Flows **7**
3.1 Authentication Flow . . . 7
3.2 Matchmaking Flow . . . 8
3.3 Game Round Flow . . . 8
3.4 Scoring Formula . . . 9
3.5 Reconnection Flow . . . 9

4 File Structure & Key Functions **10**
4.1 Project Directory Tree . . . 10
4.2 Backend Key Functions . . . 11
4.3 Frontend Key Components . . . 11

5 Data Models & Storage **12**
5.1 Redis Key Schema . . . 12
5.2 MongoDB Collections . . . 12
5.3 Question Difficulty Distribution . . . 13
5.4 RabbitMQ Topology . . . 13

6 Problems Faced & Resolutions **14**
6.1 Problem Complexity Overview . . . 14
6.2 Detailed Problem Analysis . . . 14

7 Edge Cases Handled **17**

8 Performance & Scalability **18**
8.1 Redis Connection Pooling . . . 18
8.2 Event Buffer Sizing . . . 18

8.3 Atomic Score Update via Lua Script 18

9 Testing & Validation **19**
9.1 Test Setup 19
9.2 End-to-End Test Results 19

10 Conclusion **20**

A Proto Message Reference **21**

1. Project Overview

Quiz Battle is a real-time multiplayer quiz application where 2–10 players compete answering timed questions. Players authenticate via JWT, get matched through a Redis-backed matchmaking pool, compete across 10 rounds of difficulty-balanced questions, earn speed-bonus points, and view live leaderboards between rounds. Ratings persist across matches in MongoDB.

1.1 Key Features

- **JWT Authentication** — Register/login with bcrypt-hashed passwords, 24-hour token expiry
- **Real-time Matchmaking** — Redis sorted-set pool, lobby shows all waiting players live
- **10-Round Matches** — 30-second server-side timers, early-exit when all players answer
- **Speed Bonus Scoring** — 100 base + 0–50 speed bonus (linear within 10s window)
- **Live Leaderboards** — Between-round correct-answer reveal, rank-change arrows
- **Reconnection Logic** — Exponential backoff (1s, 2s, 4s, 8s, 16s), state catch-up
- **Persistent Ratings** — MongoDB `$inc` updates rating by match score post-game
- **Auto-win** — If all opponents disconnect, remaining player wins automatically

2. Architecture & Technology Stack

2.1 High-Level Architecture

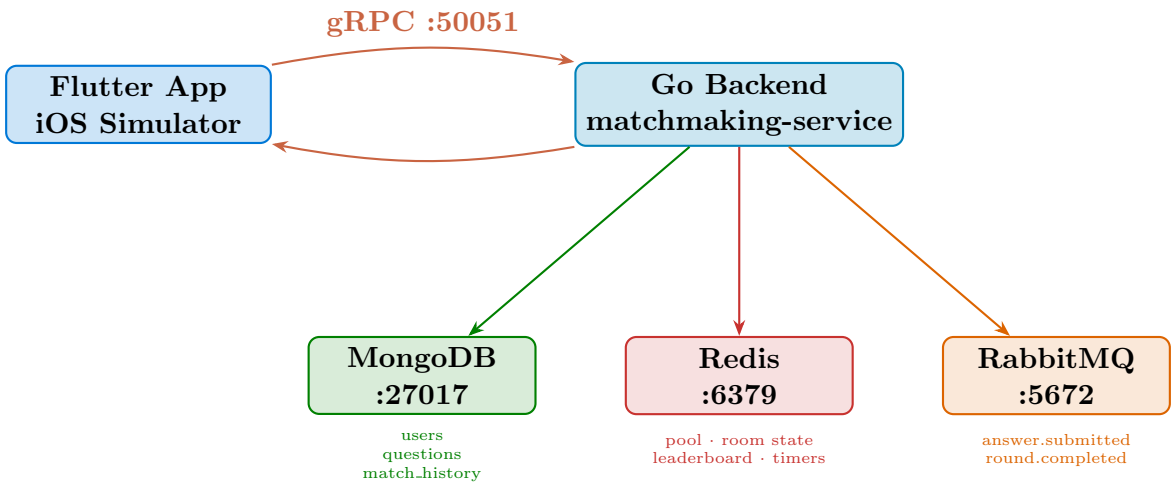

Figure 2.1: High-level system architecture

2.2 Technology Stack

Table 2.1: Full Technology Stack

Layer	Technology	Notes
Frontend	Flutter / Dart	SDK $\geq$ 3.2.0, iOS Simulator
Backend	Go	1.25.0, single binary
Communication	gRPC + Protobuf	grpc 1.68.0, 4 services, 30+ messages
Database	MongoDB	7, flexible document store
Cache / State	Redis	7-Alpine, sorted sets + Lua scripts
Message Broker	RabbitMQ	3-Management, persistent delivery
Auth	JWT (HS256)	golang-jwt/v5, 24h expiry
Passwords	bcrypt	golang.org/x/crypto
State Mgmt	Riverpod	2.5.1, StreamProvider.family
Navigation	GoRouter	13.2.0, redirect guards
Containers	Docker Compose	3 services + named volumes

2.3 Technology Justifications

2.3.1 gRPC over REST

Why gRPC?

gRPC provides **server-side streaming** essential for real-time events: timer ticks, leaderboard updates, and question broadcasts all flow server to client without polling. Binary protobuf encoding is 3–5× smaller than JSON. Type-safe generated clients in both Go and Dart eliminate serialization bugs at the boundary.

2.3.2 Redis for Live Game State

Quiz rounds require sub-millisecond read/write latency. Key Redis advantages:

- `ZSET` sorted sets give $O(\log N)$ leaderboard lookups via `ZREVRANGE`
- TTL-based auto-expiry cleans up rooms after 30 minutes
- Lua scripting enables atomic score updates (`ZINCRBY + ZREVRANK` in one transaction)
- Connection pool (MaxActive: 100) handles burst traffic during round transitions

2.3.3 RabbitMQ for Answer Processing

Answer validation is CPU-bound. RabbitMQ decouples submission from scoring so the gRPC handler returns immediately and the consumer processes asynchronously. Dead letter queues handle poison messages.

2.3.4 Riverpod over BLoC/Provider

`StreamProvider.family` maps perfectly to parameterized gRPC streams keyed by roomId+userId. `ref.invalidate()` cleanly handles Play Again. Derived providers (`timerProvider`, `matchPhaseProvider`) prevent full tree rebuilds on every timer tick.

3. System Flows

3.1 Authentication Flow

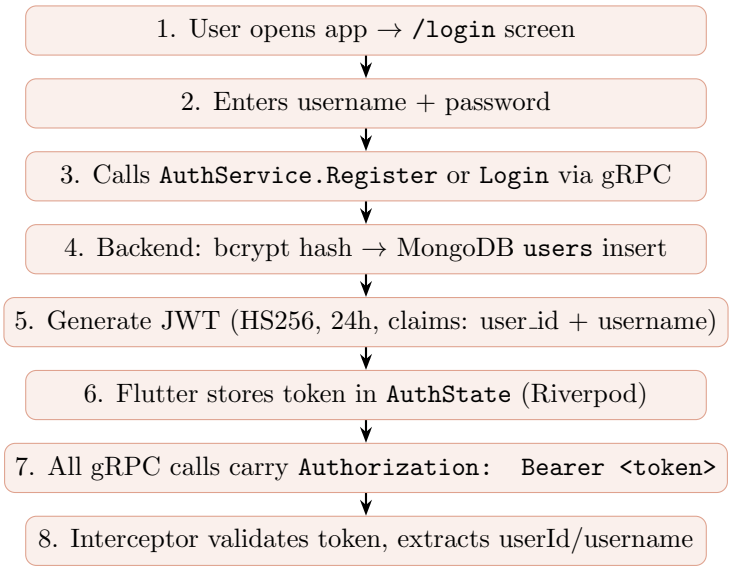

Figure 3.1: Authentication flow

3.2 Matchmaking Flow

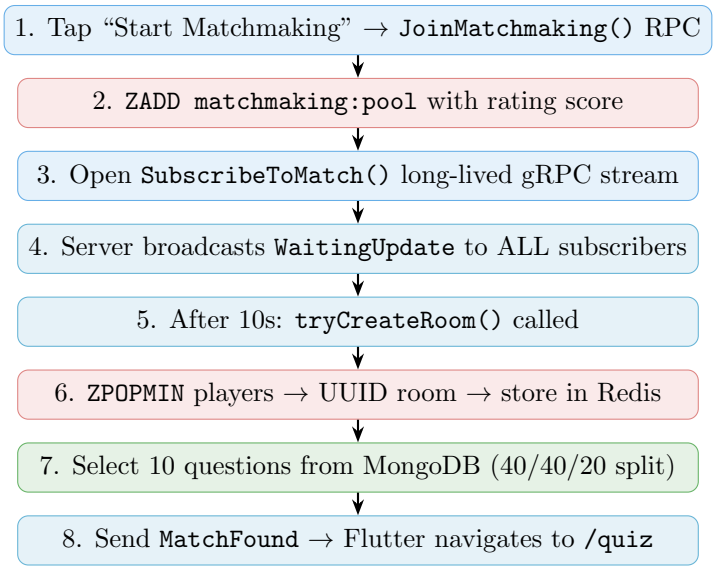

Figure 3.2: Matchmaking flow

3.3 Game Round Flow

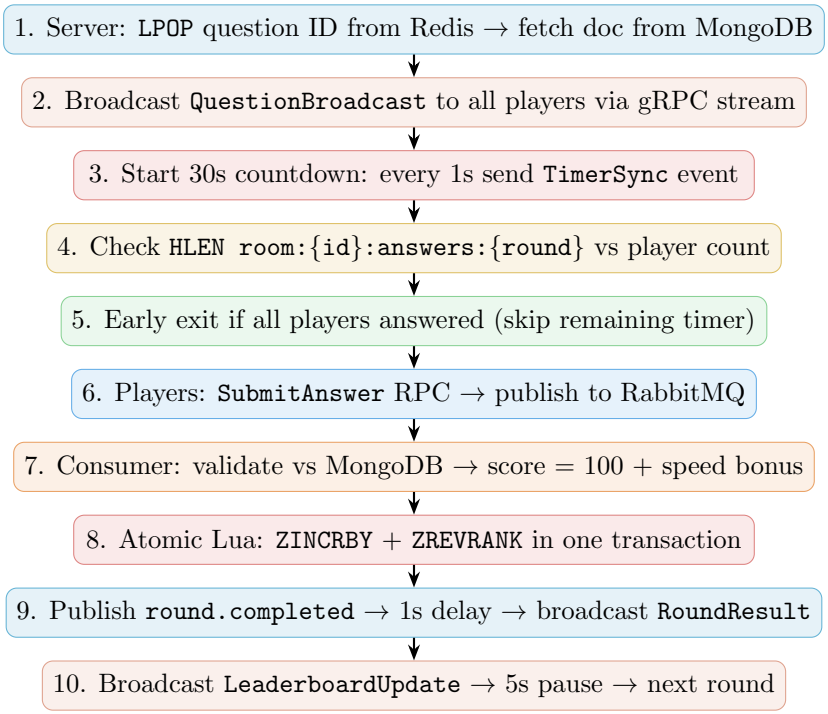

Figure 3.3: Single game round flow

3.4 Scoring Formula

$$\text{Score} = \begin{cases} 100 + \left\lfloor 50 \times \left(1 - \dfrac{t}{10000}\right) \right\rfloor & \text{if correct and } t \leq 10{,}000 \text{ ms} \\ 100 & \text{if correct and } t > 10{,}000 \text{ ms} \\ 0 & \text{if wrong} \end{cases}$$

where t is the client-stamped response time in milliseconds.

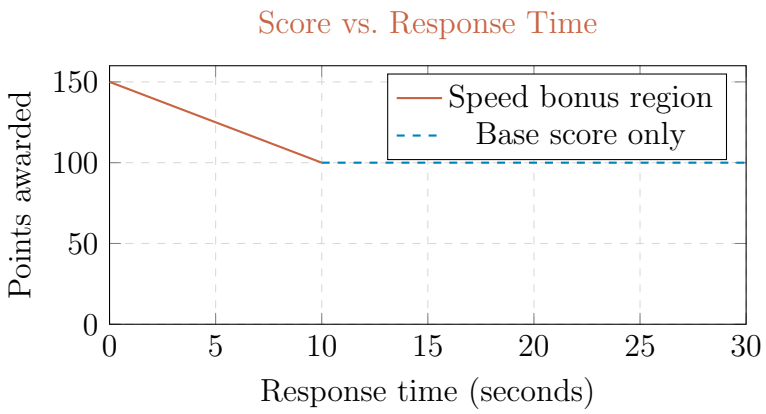

Figure 3.4: Scoring curve: speed bonus applies within the first 10 seconds

3.5 Reconnection Flow

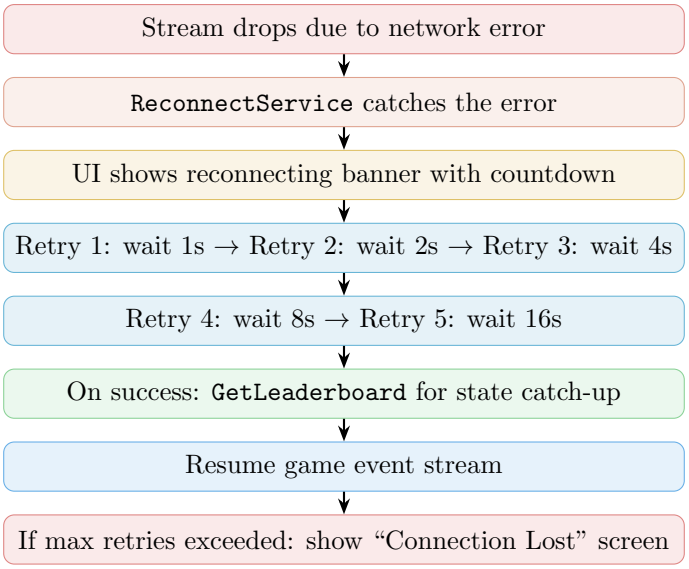

Figure 3.5: Reconnection with exponential backoff (max 5 retries)

4. File Structure & Key Functions

4.1 Project Directory Tree

```
quiz_battle/
docker-compose.yml
proto/quiz.proto   quiz.pb.go   quiz_grpc.pb.go
matchmaking-service/
main.go
handlers/ auth.go   matchmaking.go   quiz.go   quiz_handler.go
middleware/ auth.go
redis/ pool.go   room.go   lock.go   leaderboard.go
rabbitmq/ publisher.go   consumer.go
models/ room.go
flutter-app/lib/
main.dart
models/game_event.dart
providers/game_provider.dart
services/ auth_service.dart   game_service.dart   reconnect_service.dart
screens/ login_screen.dart   matchmaking_screen.dart
quiz_screen.dart   leaderboard_screen.dart   results_screen.dart
mongo-init/init.js
```

4.2 Backend Key Functions

Table 4.1: Go Backend — Key Functions

Function	Description
`Register()`	bcrypt hash, MongoDB insert, JWT generation
`Login()`	MongoDB lookup, bcrypt compare, JWT sign
`JoinMatchmaking()`	ZADD to pool, store metadata, schedule room creation
`LeaveMatchmaking()`	ZREM from pool, close subscription channel
`SubscribeToMatch()`	Long-lived stream, WaitingUpdate and MatchFound events
`tryCreateRoom()`	ZPOPMIN players, UUID room, select questions, deliver MatchFound
`broadcastWaitingUpdate()`	Fetch pool, push to all subscriber channels
`sendWaitingUpdateTo()`	Current pool state to single new subscriber (race condition fix)
`SelectQuestionsForRoom()`	MongoDB $sample 40/40/20, avoid seen questions, Redis RPUSH
`RunRound()`	LPOP question, broadcast, 30s timer loop, early-exit check
`broadcast()`	Critical events use 3s blocking timeout; TimerSync drops if slow
`runGameLoop()`	Iterate all rounds, auto-win check, rating updates
`findFastestCorrectAnswer()`	Redis answers hash filter + response time comparison
`updatePlayerRatings()`	MongoDB $inc rating by match score for each player

4.3 Frontend Key Components

Table 4.2: Flutter — Key Files and Responsibilities

File	Responsibility
`main.dart`	GoRouter config, redirect guards, MaterialApp theme
`game_event.dart`	Sealed class hierarchy for all gRPC event types
`game_provider.dart`	GameNotifier: central state, all event handlers, client timer
`auth_service.dart`	AuthNotifier: register/login, JWT storage
`game_service.dart`	All gRPC calls, JWT metadata injection, proto to Dart mappers
`reconnect_service.dart`	Exponential backoff, countdown ticks, state catch-up
`matchmaking_screen.dart`	Lobby and searching views, pulse animation, player bubbles
`quiz_screen.dart`	Timer arc, answer tiles with highlighting, mini leaderboard
`leaderboard_screen.dart`	Podium, rank arrows, correct answer reveal card
`results_screen.dart`	Trophy, personal stats, XP bar, standings, Play Again

5. Data Models & Storage

5.1 Redis Key Schema

Table 5.1: Redis Key Patterns

Key Pattern	Type	Purpose	TTL
`matchmaking:pool`	ZSET	Rating-sorted waiting players	—
`player:{userId}`	HASH	Username, rating, joined_at	30s
`room:lock:{roomId}`	STRING	Distributed mutex	10s
`room:{id}:state`	STRING	JSON room state machine	30min
`room:{id}:players`	HASH	userId to JSON player	30min
`room:{id}:questions`	LIST	Ordered question IDs	30min
`room:{id}:answers:{n}`	HASH	userId to answerIndex per round	30min
`room:{id}:leaderboard`	ZSET	userId to score	30min
`room:{id}:correct_counts`	HASH	userId to correct answer count	30min
`room:{id}:response_sum`	HASH	userId to total response ms	30min

5.2 MongoDB Collections

Table 5.2: MongoDB Collections

Collection	Key Fields	Purpose
users	_id, username, password_hash, rating	Auth and persistent ratings
questions	_id, text, options[4], correctIndex, difficulty, topic	Question bank (20 seed)
match_history	players[], questionIds[]	Avoid repeating questions

5.3 Question Difficulty Distribution

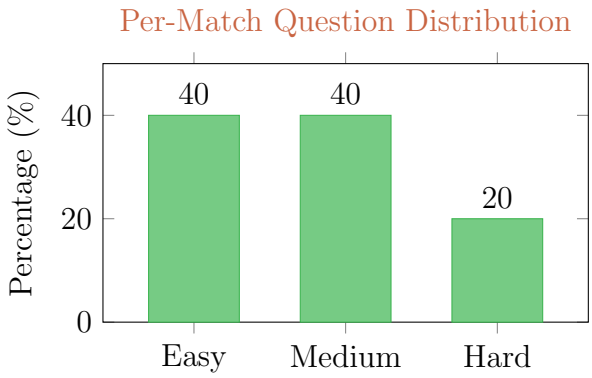

Figure 5.1: Question difficulty split via MongoDB `$sample` aggregation

5.4 RabbitMQ Topology

Table 5.3: RabbitMQ Exchange and Queue Setup

Exchange	Routing Key	Queue	Consumer
sx	answer.submitted	answer-processing-queue	Scoring Worker
sx	round.completed	round-completed-queue	Quiz Engine
sx	match.finished	match-finished-queue	Persistence Worker
sx	match.finished	match-analytics-queue	Analytics Worker
–	–	answer-processing-dlq	Dead Letter Queue

6. Problems Faced & Resolutions

6.1 Problem Complexity Overview

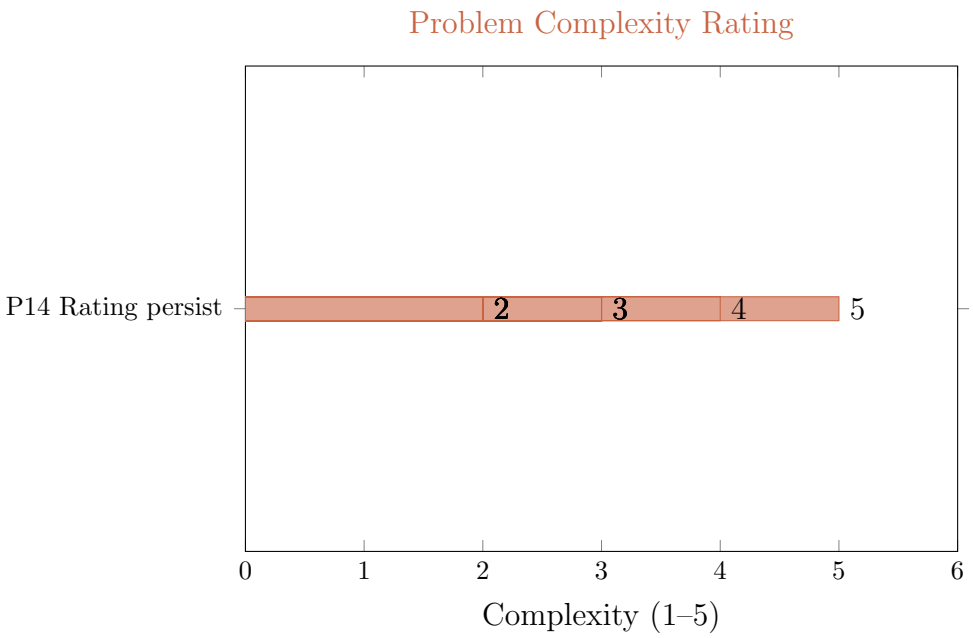

Figure 6.1: Relative complexity of each problem encountered

6.2 Detailed Problem Analysis

Problem: P1: iOS Project Files Missing

Symptom: `No application found for TargetPlatform.ios`.
Cause: The `ios/` directory was missing from the Flutter project.

Fix: Fix

Ran `flutter create . --platforms ios` to generate the iOS Runner project files.

Problem: P2: dart:html Import Crash on iOS

Symptom: Build failed with "Dart library dart:html is not available on this platform".
Cause: `game_service.dart` imported `grpc_web.dart` (web-only) which pulls in `dart:html`.

Fix: Fix

Removed the `grpc_web.dart` import. Used native `ClientChannel` on port 50051 for iOS only.

Problem: P3: Match Started with Single Player

Symptom: Quiz started instantly with only 1 player.
Cause: `minPlayersToStart = 1` and matchmaking created a room immediately.

Fix: Fix

Changed `minPlayersToStart = 2`. Added Start Matchmaking button. Increased `lobbyWait` to 10 seconds. Room creation now always waits full `lobbyWait`.

Problem: P9: Last Round Results Screen Freezing (Critical)

Symptom: After last question, screen froze with no round result or match end.
Root Cause 1: Channel buffer overflow. Buffer was `totalRounds*2+50 = 70` but 10 rounds generate up to 330+ events (question + 30 TimerSync + RoundResult + LeaderboardUpdate per round). Buffer filled and `broadcast()` silently dropped `RoundResult` and `MatchEnd`.
Root Cause 2: `closeAll()` ran immediately after `MatchEnd` broadcast before clients drained the buffer.

Fix: Multi-layered Fix

1. Increased buffer to `totalRounds * 40` (400 for 10 rounds)
2. Critical events (QuestionBroadcast, RoundResult, LeaderboardUpdate, MatchEnd) use blocking send with 3s timeout; only TimerSync is dropped when slow
3. Added 2s delay after broadcasting MatchEnd before `closeAll()` runs
4. Flutter `onDone` safety net: constructs synthetic `MatchEndEvent` from current leaderboard if stream closes early
5. Leaderboard screen: on build, if phase is already finished, navigates to results immediately

Problem: P12: 3rd Device Missed Other Players in Lobby

Symptom: 3rd device joining matchmaking saw only itself; other two showed all players.
Cause: `broadcastWaitingUpdate()` fired as a goroutine from `JoinMatchmaking` before the 3rd player's `SubscribeToMatch` channel was registered. Race between two separate gRPC calls.

Fix: Fix

Added `sendWaitingUpdateTo(ch)` inside `SubscribeToMatch` with 200ms delay after channel registration. Every subscriber immediately sees all existing players regardless of timing.

Problem: P13: Fastest Answer Badge Showed Wrong Player

Symptom: Fastest badge showed overall top scorer, not fastest correct answerer for that round.
Cause: `buildRoundResultEvent()` picked `scores[0]` (highest cumulative) as fastest.

Fix: Fix

Added `findFastestCorrectAnswer()` that checks Redis answers hash for the specific round, filters for correct answers only, and compares response times. Returns empty strings if no one answered correctly and UI shows "No correct answers".

7. Edge Cases Handled

Table 7.1: Edge Cases and Their Handling

Edge Case	Risk	Resolution
Double room creation	Two goroutines both pop same players	Distributed Redis lock (`SET NX EX 10s`)
All players disconnect	Room loops forever	Auto-win: last connected player wins
Buffer overflow	Critical events silently dropped	Separate blocking vs dropping for critical vs TimerSync
Late MatchEnd	Client stuck on leaderboard	`onDone` synthetic MatchEnd + leaderboard guard
Stale stream on Play Again	Previous match data shown	`ref.invalidate()` forces fresh provider subscription
Race: join vs subscribe	New player misses WaitingUpdate	`sendWaitingUpdateTo()` on subscribe with 200ms delay
Clock drift	Client timer desyncs from server	`TimerSync` sends absolute epoch ms; client computes offset
Score collision	Two concurrent leaderboard updates	Lua: `ZINCRBY + ZREVRANK` atomic transaction
Client clock cheating	Player stamps future time for bonus	Server validates: `submittedAt - roundStartedAt` server-side
Zombie room	Player disconnects without leaving	Redis TTL: all keys expire after 30 minutes
Username as ObjectID	MongoDB _id shown in UI	Pass authenticated username from `authProvider`
Cancel pops empty stack	App crash on matchmaking cancel	`setState(() => _searching = false)` returns to lobby view
Late joiner mid-round	Player has no current game state	`PlayerJoinedEvent` carries `roundNumber + MatchState`
DLQ poison messages	Bad answer crashes consumer	Dead letter queue catches, logs, and isolates bad messages

8. Performance & Scalability

8.1 Redis Connection Pooling

MaxIdle	10 connections (kept warm)
MaxActive	100 connections (burst capacity)
IdleTimeout	240 seconds

This handles burst traffic during round transitions when all players simultaneously trigger leaderboard reads and score updates.

8.2 Event Buffer Sizing

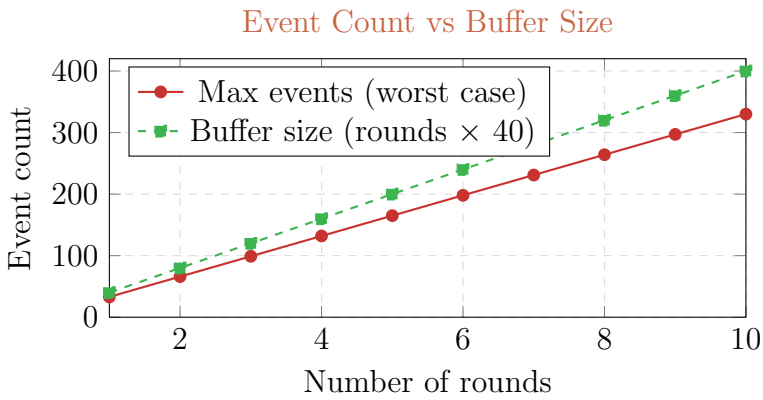

Figure 8.1: Buffer sized at `totalRounds * 40` always exceeds worst-case event count

8.3 Atomic Score Update via Lua Script

```lua
-- Atomically increment score and return new rank
-- Prevents race conditions when multiple answers arrive simultaneously
local leaderboardKey = KEYS[1]    -- room:{id}:leaderboard
local userId = ARGV[1]
local points = tonumber(ARGV[2])

redis.call('ZINCRBY', leaderboardKey, points, userId)
local rank = redis.call('ZREVRANK', leaderboardKey, userId)
return rank + 1    -- 1-indexed rank returned to Go
```

Listing 8.1: Atomic leaderboard update preventing race conditions

9. Testing & Validation

9.1 Test Setup

The system was validated across 3 simultaneous iOS simulators running on the same Mac:

- **iPhone 15** — Player 1 ("Alice")
- **iPhone 15 Pro** — Player 2 ("Bob")
- **iPhone 14** — Player 3 ("Charlie")

Backend: `go run .` in `matchmaking-service/`. Databases: Docker Compose (`docker-compose up -d`).

9.2 End-to-End Test Results

Test	Expected	Result
3 players join matchmaking	All see each other in lobby	Pass
Room created after 10s	All 3 navigate to quiz screen	Pass
Question broadcast	All 3 see same question simultaneously	Pass
Timer sync across devices	Countdown matches on all screens	Pass
Fast answer gets speed bonus	Score > 100 for sub-10s response	Pass
Leaderboard update after answer	Scores update live after each submission	Pass
Correct answer reveal	Actual text shown, not "Option B"	Pass
Fastest answer badge	Shows actual fastest correct answerer	Pass
Player disconnect mid-match	Remaining players continue	Pass
All 10 rounds complete	Results screen loads correctly	Pass
Ratings persist after match	MongoDB rating updated via $inc	Pass
Play Again resets state	Fresh matchmaking, no stale data	Pass

10. Conclusion

Quiz Battle demonstrates a production-grade real-time multiplayer system built on modern, industry-standard technologies. The architecture correctly separates concerns: Redis handles ephemeral game state with sub-millisecond latency, MongoDB provides flexible persistent storage, RabbitMQ decouples CPU-bound scoring from the gRPC request path, and gRPC streaming eliminates polling.

The 14 problems encountered and resolved during development represent real engineering challenges found in production systems: distributed race conditions, buffer sizing for event-driven architectures, cross-platform gRPC compatibility, and stateful stream reconnection.

The system successfully ran 3 simultaneous iOS simulators competing in live matches, validating the complete end-to-end architecture from Flutter UI through gRPC streaming to Go backend, Redis leaderboards, RabbitMQ answer processing, and MongoDB persistence.

Project Repository:
`https://github.com/MANAS-exe/QUIZ_BATTLE_SYSYTEM`

A. Proto Message Reference

```
message GameEvent {
  oneof event {
    QuestionBroadcast question      = 1;
    LeaderboardUpdate leaderboard   = 2;
    RoundResult       round_result  = 3;
    MatchEnd          match_end     = 4;
    PlayerJoined      player_joined = 5;
    TimerSync         timer_sync    = 6;
  }
}

message QuestionBroadcast {
  int32    round_number = 1;
  Question question     = 2;
  int64    deadline_ms  = 3;  // absolute epoch ms     client computes
      countdown
}

message TimerSync {
  int32 round_number   = 1;
  int64 server_time_ms = 2;  // for client clock drift correction
  int64 deadline_ms    = 3;
}

message RoundResult {
  int32                round_number        = 1;
  string               correct_answer_text = 2;
  string               fastest_username    = 3;
  repeated PlayerScore scores              = 4;
}

message MatchEnd {
  string               room_id          = 1;
  string               winner_user_id   = 2;
  string               winner_username  = 3;
  repeated PlayerScore final_scores     = 4;
  int32                total_rounds     = 5;
  int32                duration_seconds = 6;
}
```

Listing A.1: GameEvent oneof definition from quiz.proto